

Edge cases (fix a flaky test) — Solution

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Story — `llm_output_testing_practice.py` (Edge cases section): everyone writes `assert response == "..."` against a real model once, and watches it fail on a perfectly good answer. Fixing one such test on purpose teaches the habit every later LLM test needs: check what makes the answer correct, not its exact words. **If `not`:** the Build Task's rule-based tasks would be written as exact matches, fail at random, and get ignored.

Every version calls the real OpenAI API, and runs from inside `practice/` with `pytest llm_output_testing_practice.py -v`. Read both depths — they're not "wrong, right," they're 2 real, valid ways to fix the same test, with real tradeoffs between them.

Basic Version

Approach 1 — the direct way

```
# practice/llm_output_testing_practice.py – Edge cases section
from openai import OpenAI

client = OpenAI()

SHIPPING_PROMPT = (
    "You are a support assistant. Order A123 shipped today.\n"
    "Customer: Has my order shipped?"
)

def call_model(prompt):
    response = client.chat.completions.create(
        model="gpt-4o-mini",
        temperature=0,
        messages=[{"role": "user", "content": prompt}],
    )
    return response.choices[0].message.content

# broken (flaky) – kept here only to compare; delete it once you've
# seen it fail on a correct answer
```

```
def test_shipping_reply_exact():
    response = call_model(SHIPPING_PROMPT)
    assert response == "Your order has shipped."

# fixed
def test_shipping_reply():
    response = call_model(SHIPPING_PROMPT)
    assert "shipped" in response.lower()
    assert len(response.split()) < 60
```

Expected output (the model's exact words vary from run to run):

```
llm_output_testing_practice.py::test_shipping_reply_exact FAILED
llm_output_testing_practice.py::test_shipping_reply PASSED
```

The exact-match test fails on a correct answer like "Yes, your order A123 shipped today!". The fixed version checks the two things that actually matter: the key fact is there, and the reply is short.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Approach 1 — plain property checks, each with a reason

Story: a property check is only as good as the property. Writing the reason next to each `assert` forces you to say *why* it proves correctness, and the failure message shows the real reply. **If not:** a failing test would say only `AssertionError`, and you'd rerun the model by hand to see what it said.

```
# practice/llm_output_testing_practice.py – Edge cases section
from openai import OpenAI

client = OpenAI()

SHIPPING_PROMPT = (
    "You are a support assistant. Order A123 shipped today.\n"
    "Customer: Has my order shipped?"
)

def call_model(prompt: str) -> str:
    response = client.chat.completions.create(
        model="gpt-4o-mini",
        temperature=0,
        messages=[{"role": "user", "content": prompt}],
    )
    return response.choices[0].message.content

def test_shipping_reply():
    response = call_model(SHIPPING_PROMPT)
    # why: the key fact must be present, whatever the wording
    assert "shipped" in response.lower(), response
    # why: a direct reply, not a ramble
    assert len(response.split()) < 60, response
```

Expected output:

```
llm_output_testing_practice.py::test_shipping_reply PASSED
```

Approach 2 — one of several correct phrasings

Story: "shipped" isn't the only correct word — "on its way" and "dispatched" are just as right. A single-keyword check fails on those and becomes a *new* flaky test. **If not:** you'd swap one kind of random failure for another, and the test would still get ignored.

```
# practice/llm_output_testing_practice.py – Edge cases section
from openai import OpenAI

client = OpenAI()

SHIPPING_PROMPT = (
    "You are a support assistant. Order A123 shipped today.\n"
    "Customer: Has my order shipped?"
)

# why: every wording that correctly means "it has shipped"
ACCEPTABLE_PHRASES = ["shipped", "on its way", "sent out", "dispatched"]

def call_model(prompt: str) -> str:
    response = client.chat.completions.create(
        model="gpt-4o-mini",
        temperature=0,
        messages=[{"role": "user", "content": prompt}],
    )
    return response.choices[0].message.content

def test_shipping_reply():
    response = call_model(SHIPPING_PROMPT).lower()
    found = False
    for phrase in ACCEPTABLE_PHRASES:
        # how: one match is enough – any correct wording passes
        if phrase in response:
            found = True
    assert found, "no shipping phrase found in: " + response
    assert len(response.split()) < 60, response
```

Expected output:

```
llm_output_testing_practice.py::test_shipping_reply PASSED
```

If this ever fails, read the reply in the failure message. If it's a correct answer in a wording you didn't list, add the wording. If it's a wrong answer, the test did its job.

Difference from Basic: Approach 1 keeps Basic's two property checks but adds a reason for each and shows the real reply when one fails. Approach 2 accepts any of several correct wordings, so the fix doesn't create a new kind of flakiness on answers that are right but worded differently.

Which one should you actually write? Approach 2's phrase list for any fact with more than one natural wording — which is most of them. Approach 1's single keyword is fine when there's really only one way to say it (an order number, a price). The Build Task's `run_eval.py` uses the same idea: its exact-check tasks look for a short `expected` phrase inside the answer, never the whole answer.